

Question 2

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("Dataset_Day7.csv")
temp = df.copy()

# Identify missing values (where 0 is considered missing)
missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction"]
MV = (temp[missing_cols] == 0).sum()

print("% of Missing values are present in:")
print((MV[MV > 0] / len(temp)) * 100)

# Replace 0s with NaN and fill with median
for col in MV[MV > 0].index:
    temp[col] = temp[col].replace(0, np.nan)
    temp[col] = temp[col].fillna(temp[col].median())

# Save cleaned data
df_NoMV = temp.copy()

# Check for remaining missing values
print("\nMissing values after cleaning:")
print(df_NoMV[missing_cols].isna().sum())

# Z-score standardization
features_to_scale = ["Pregnancies", "Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
for col in features_to_scale:
    df_NoMV[col] = (df_NoMV[col] - df_NoMV[col].mean()) / df_NoMV[col].std()

# Outlier detection using Z-score method
outlier_condition = (np.abs(df_NoMV[features_to_scale]) > 3).any(axis=1)
OutlierRows = df_NoMV[outlier_condition]

print("\n% of Outlier rows in the dataset is {:.2f}%\n".format(len(OutlierRows) / len(df_NoMV) * 100))

# Remove outliers
df_NoMV_OutlierFree = df_NoMV.drop(OutlierRows.index, axis=0)

# Final dataset info
df_NoMV_OutlierFree.info()
```

```
(myenv) flara_04@flara: /mnt/d/python/day7 $ python3 q1.py
% of Missing values are present in:
Glucose      0.651042
BloodPressure 4.557292
BMI           1.432292
dtype: float64

Missing values after cleaning:
Glucose      0
BloodPressure 0
BMI           0
DiabetesPedigreeFunction 0
dtype: int64

% of Outlier rows in the dataset is 3.91%

<class 'pandas.core.frame.DataFrame'>
Index: 738 entries, 0 to 767
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies            738 non-null   float64
Index: 738 entries, 0 to 767
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies            738 non-null   float64
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies            738 non-null   float64
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies            738 non-null   float64
1   Glucose                 738 non-null   float64
2   BloodPressure           738 non-null   float64
3   BMI                     738 non-null   float64
4   DiabetesPedigreeFunction 738 non-null   float64
5   Age                     738 non-null   float64
6   Outcome                 738 non-null   int64
dtypes: float64(6), int64(1)
memory usage: 46.1 KB
(myenv) flara_04@flara: /mnt/d/python/day7 $
```

```

import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("Dataset_Day7.csv")
temp = df.copy()

# Identify missing values (where 0 is considered missing)
missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction"]
MV = (temp[missing_cols] == 0).sum()

print("% of Missing values are present in:")
print((MV[MV > 0] / len(temp)) * 100)

# Replace 0s with NaN and fill with median
for col in MV[MV > 0].index:
    temp[col] = temp[col].replace(0, np.nan)
    temp[col] = temp[col].fillna(temp[col].median())

# Save cleaned data
df_NoMV = temp.copy()

# Check to confirm there are no missing values
print("\nMissing values after cleaning:")
print(df_NoMV[missing_cols].isna().sum())

```

```

% of Missing values are present in:
Glucose      0.651042
BloodPressure 4.557292
BMI           1.432292
Glucose      0.651042
BloodPressure 4.557292
BMI           1.432292
BloodPressure 4.557292
BMI           1.432292
dtype: float64
BMI           1.432292
dtype: float64
dtype: float64

Missing values after cleaning:
Glucose      0
BloodPressure 0
BMI           0
DiabetesPedigreeFunction 0
dtype: int64

```

```

day7 > q3.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
6
7 # Load dataset
8 df = pd.read_csv("Dataset_Day7.csv")
9 temp = df.copy()
10
11 # Handle missing values
12 missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction"]
13 for col in missing_cols:
14     temp[col] = temp[col].replace(0, np.nan)
15     temp[col] = temp[col].fillna(temp[col].median())
16
17 df_NoMV = temp.copy()
18
19 # Standardize features
20 features_to_scale = ["Pregnancies", "Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
21 for col in features_to_scale:
22     df_NoMV[col] = (df_NoMV[col] - df_NoMV[col].mean()) / df_NoMV[col].std()
23
24 # Remove outliers using Z-score
25 outlier_condition = (np.abs(df_NoMV[features_to_scale]) > 3).any(axis=1)
26 df_NoMV_OutlierFree = df_NoMV[~outlier_condition] # Keep only non-outliers
27
28 # ✅ Now continue with Q3
29 X = df_NoMV_OutlierFree.drop("Outcome", axis=1)
30 y = df_NoMV_OutlierFree["Outcome"]
31
32 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
33
34 print("✅ Q3: Data split completed (70% train, 30% test).")
35 print(f"Training set size: {len(X_train)} rows")
36 print(f"Testing set size: {len(X_test)} rows\n")
37
38 model = LogisticRegression(max_iter=1000)
39 model.fit(X_train, y_train)
40 y_pred = model.predict(X_test)
41
42 accuracy = accuracy_score(y_test, y_pred)
43 precision = precision_score(y_test, y_pred)
44 recall = recall_score(y_test, y_pred)
45 f1 = f1_score(y_test, y_pred)
46
47 print("✅ Q3a: Model trained and evaluated.")
48 print(f"Accuracy : {accuracy:.4f}")
49 print(f"Precision: {precision:.4f}")
50 print(f"Recall   : {recall:.4f}")
51 print(f"F1 Score : {f1:.4f}")
52

```

```

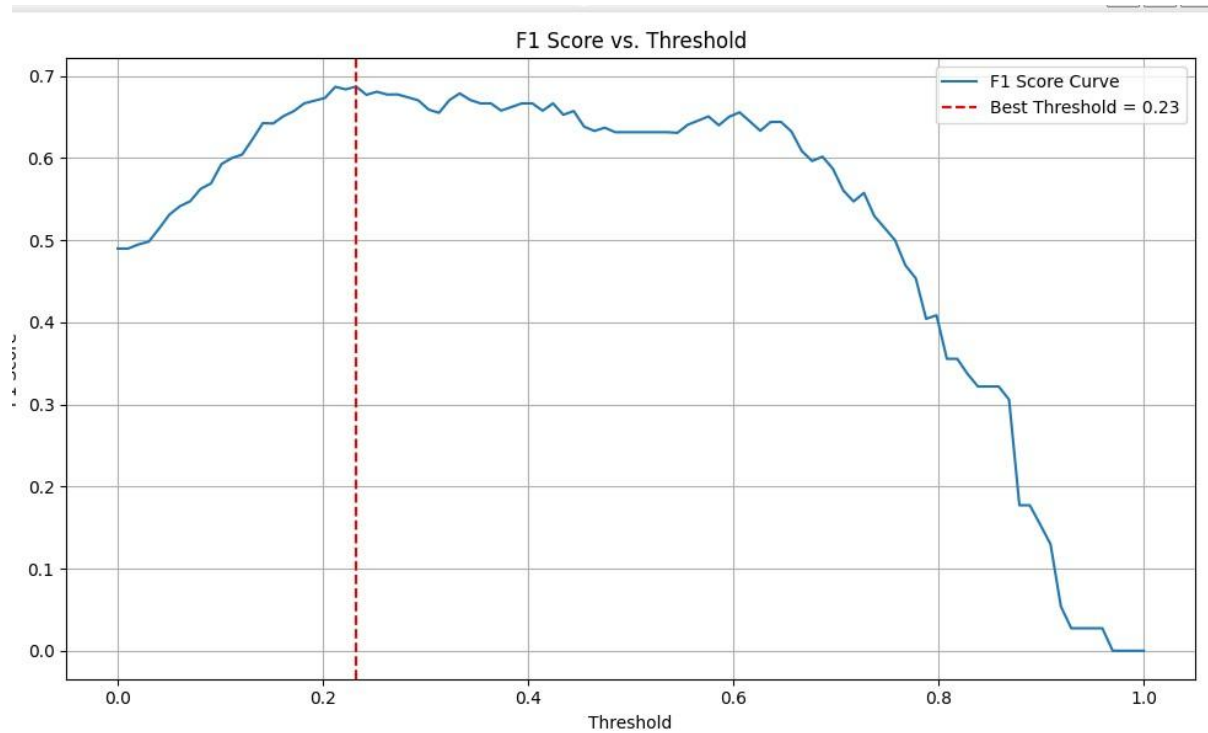
✅ Q3a: Model trained and evaluated.
✅ Q3: Data split completed (70% train, 30% test).
Training set size: 516 rows
Testing set size: 222 rows

✅ Q3a: Model trained and evaluated.
Accuracy : 0.7793
Precision: 0.6885
Recall   : 0.5833
F1 Score : 0.6316
(myenv) flora_04@Flora: /mnt/d/python/day7$

```

```
# Plot F1 Score vs Threshold
plt.figure(figsize=(10, 6))
plt.plot(thresholds, f1_scores, label='F1 Score Curve')
plt.axvline(best_threshold, color='red', linestyle='--', label=f'Best Threshold = {best_threshold:.2f}')
plt.xlabel("Threshold")
plt.ylabel("F1 Score")
plt.title("F1 Score vs. Threshold")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Output
print("✅ Q4: F1 Score vs. Threshold plot completed.")
print(f"Best Threshold: {best_threshold:.3f}")
print(f"Best F1 Score: {best_f1_score:.4f}")
```




```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Load and Preprocess Data
df = pd.read_csv("Dataset_Day8.csv")
temp = df.copy()

# Handle missing values: Replace 0 with NaN, then fill with median
missing_cols = ["Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction"]
for col in missing_cols:
    temp[col] = temp[col].replace(0, np.nan)
    temp[col] = temp[col].fillna(temp[col].median())

# Z-score normalization
features = ["Pregnancies", "Glucose", "BloodPressure", "BMI", "DiabetesPedigreeFunction", "Age"]
temp[features] = (temp[features] - temp[features].mean()) / temp[features].std()

# Remove outliers (Z-score > 3)
outlier_mask = (np.abs(temp[features]) > 3).any(axis=1)
df_cleaned = temp[~outlier_mask]

# Split Data
X = df_cleaned.drop("Outcome", axis=1)
y = df_cleaned["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)

# Plot Precision & Recall vs k (Euclidean)
k_range = range(1, 31)
precisions = []
recalls = []
f1_scores = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k, metric='euclidean')
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)

    precisions.append(precision_score(y_test, y_pred))
    recalls.append(recall_score(y_test, y_pred))
    f1_scores.append(f1_score(y_test, y_pred))

# Identify best k
best_k = k_range[np.argmax(f1_scores)]
best_f1 = max(f1_scores)

# Plot Precision & Recall vs. k
plt.figure(figsize=(10, 6))
plt.plot(k_range, precisions, label='Precision', marker='o')
plt.plot(k_range, recalls, label='Recall', marker='s')
plt.axvline(x=best_k, color='r', linestyle='--', label=f'Best k (F1) = {best_k}')
print(f"✅ Best k (F1-Score) using Euclidean distance: k = {best_k}, F1 = {best_f1:.4f}")

# Best k and Distance Metric Combination

distance_metrics = ['euclidean', 'manhattan', 'chebyshev', 'minkowski']
best_combo = None
best_f1_c = 0

for metric in distance_metrics:
    for k in range(1, 31):
        knn = KNeighborsClassifier(n_neighbors=k, metric=metric)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        current_f1 = f1_score(y_test, y_pred)
        if current_f1 > best_f1_c:
            best_f1_c = current_f1
            best_combo = (k, metric)

print("\n✅ Best combination for kNN:")
print(f"Best k = {best_combo[0]}")
print(f"Best distance metric = '{best_combo[1]}'")

```

✓ Best k (F1-Score) using Euclidean distance: $k = 21$, $F1 = 0.6154$

✓ Best combination for kNN:

Best $k = 21$

Best distance metric = 'euclidean'

Best F1 Score = 0.6154

